Вот несколько примеров общих вопросов, не связанных напрямую с исходным кодом, на которые вы должны быть в состоянии ответить и обсудить при сдачи задания:

- Что мы подразумеваем под конвейером?
- Что такое программный канал?
- Как каналы используются для построения конвейеров?
- Покажите пример конвейера процессов в терминале (shell).
- Как реализуются команды оболочки (терминала)?
- Что происходит с таблицей дескрипторов файлов после успешного создания нового конвейера?

- Сколько раз родитель вызывает fork и почему?
- Почему дети должны вызывать execlp()?
- Объясните, как каждый ребенок может перенаправить stidin или stdout из канала или в канал?
- Как потребитель узнает, когда больше не надо ожидать данных из канала?
- Почему важно, чтобы процесс закрывал все дескрипторы файлов канал, которые он не собирается использовать?
- Что может произойти, если вы закроете дескриптор чтения раньше времени?

4. Задача 1 - доработка примера

При запуске готовой программы вы должны увидеть вывод, похожий на этот.

Убедитесь, что вы снова получили приглашение оболочки $.

```
osc@ubuntu:/media/sf_osc/uu$ gcc pipeline.c -o p14
osc@ubuntu:/media/sf_osc/uu$ ./p14
 CHILD_A <4561> Мой родитель <4560>
 CHILD_B <4562> Мой родитель <4560>
     1  child_8.c*
     2  dir/
     3  execlp_1.c*
     4  execv_3.c*
     5  execvp_2.c*
     6  fork_4.c*
     7  fork_5.c*
     8  fork_6.c*
     9  fork_7.c*
    10  fork_9.c*
    11  p1*
    12  p10*
    13  p11*
    14  p11a*
    15  p11b*
    16  p12*
    17  p12a*
    18  p13*
    19  p14*
    20  p2*
    21  p3*
    22  p4*
    23  p5*
    24  p6*
    25  p7*
    26  p8*
    27  p9*
    28  pipe_1a.c*
    29  pipe_1b.c*
    30  pipe_1.c*
    31  pipe_2a.c*
    32  pipe_2.c*
    33  pipe_3.c*
    34  pipeline.c*
    35  signals.c*
PARENT <4560> Goodbye!
osc@ubuntu:/media/sf_osc/uu$
```

```
vboxuser@Linux:~$ gcc 5_Qv_1.c
vboxuser@Linux:~$ ./a.out
CHILD_A <195923> Мой родитель <195922>
CHILD_B <195924> Мой родитель <195922>
     1  5_Qv_1.c
     2  6qv.c
     3  a.out*
     4  Copy_File.c
     5  Desktop/
     6  Documents/
     7  Downloads/
     8  f2.txt
     9  f4.txt
    10  f6.txt
    11  Music/
    12  Os_5.c
    13  parser.c
    14  parser.h
    15  Pictures/
    16  Public/
    17  Qv_5_2*
    18  Qv_5_2.c
    19  Qv_5_2 (Copy).c
    20  snap/
```

5. Задача2 – дописать программу оболочки

Интерфейс оболочки выдает пользователю приглашение, после которого вводится следующая команда. В примере ниже показано приглашение osh> и следующая команда пользователя: cat prog.c. (Эта команда выводит файл prog.c на экран с помощью команды UNIX cat).

```
osh>cat prog.c
```

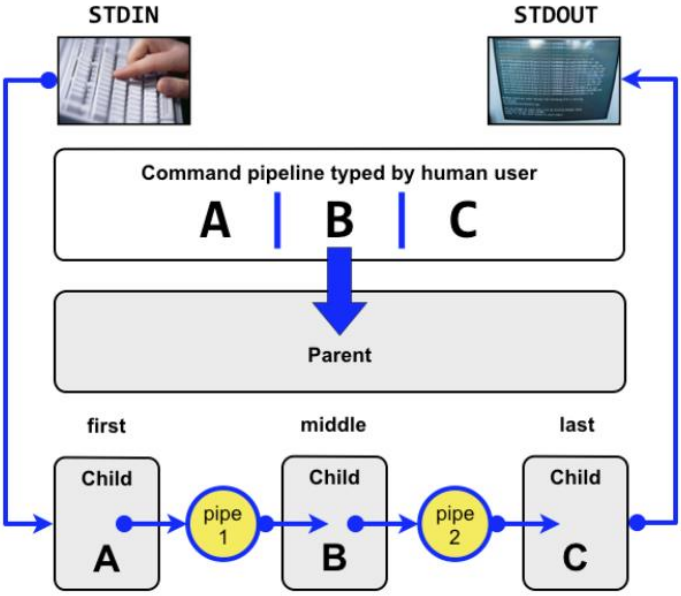
5. Задача2 – дописать программу оболочки

Дополнительно:

Модифицировать программу интерфейса оболочки таким образом, чтобы пользователь мог выполнить последнюю команду, введя !!. Например, если пользователь вводит команду ls -l, он может выполнить ее снова, введя !! в приглашении. Любая команда, выполненная таким образом должна быть отражена на экране пользователя, и команда также должна быть помещена в буфер истории как следующая команда.
Ваша программа также должна обеспечивать базовую обработку ошибок. Если в истории нет последней введенной команды, ввод !! должен приводить к появлению сообщения "Нет команд в истории".

5. Задача2 – дописать программу оболочки

На рисунке показана общая структура оболочки:



6. Задача3 – fork-бомба

По материалам лекции **5-os-2026-0306.pptx**

Добиться вывода

fork: Resource temporarily unavailable

в своей системе (os_L5_1.c)